Programación II

Guía 2

Vectores, matrices, cadenas de caracteres y punteros en C

Plataformas: Windows 10/11 y GNU/Linux
Herramientas: Visual Studio Code, GCC y GDB

Edición 2026

Propósito de esta guía

En la Guía 1 se trabajó con tipos de datos básicos, operadores, decisiones e iteraciones. En esta segunda guía práctica se ampliarán esas herramientas para comenzar a manipular **caracteres, cadenas, vectores, matrices y punteros** en C.

La intención de los ejercicios es que se pueda relacionar estructuras de datos ya conocidas con su representación en memoria y con las operaciones que el lenguaje C permite realizar sobre ellas.

A lo largo de la guía se continuará utilizando el mismo flujo de trabajo configurado en Visual Studio Code: crear un archivo fuente, compilarlo con GCC, ejecutarlo desde la terminal integrada y analizar su comportamiento cuando sea necesario.

Resultados de aprendizaje

Al finalizar podrá:

- trabajar con caracteres y reconocer su representación mediante códigos ASCII;
- recorrer y transformar cadenas de caracteres;
- utilizar funciones básicas de la biblioteca `string.h`;
- declarar, recorrer y operar con vectores y matrices;
- interpretar la relación entre un vector y un puntero;
- utilizar los operadores de dirección (&) y contenido (*) para obtener direcciones de memoria y acceder al contenido de las posiciones apuntadas;
- analizar el efecto de distintas operaciones con punteros antes de ejecutar un programa.

Antes de comenzar

Para resolver los ejercicios utiliza una carpeta general denominada, por ejemplo, `Guia02`. Guarda cada ejercicio en un archivo independiente:

```
ejercicio01.c
ejercicio02.c
ejercicio03.c
...
ejercicio10.c
```

La carpeta completa debe abrirse desde Visual Studio Code para reutilizar la configuración de compilación y depuración de la carpeta `.vscode`.

Flujo de trabajo

Para cada ejercicio se recomienda:

1. analizar la consigna antes de comenzar a programar;
2. crear el archivo fuente correspondiente;
3. escribir y guardar el programa con `Ctrl+S`;
4. compilar con `Ctrl+Shift+B`;
5. corregir los errores o advertencias informados por GCC;
6. ejecutar el programa desde la terminal integrada;

7. probarlo con distintos datos de entrada.

Cuando sea necesario comprender qué ocurre en memoria, utiliza también el depurador mediante `F5`.

Bloque A: Caracteres y cadenas de caracteres

En este bloque se trabajará con caracteres individuales y cadenas, recorriéndolos y transformándolos mediante operaciones elementales y funciones de biblioteca.

Ejercicio 1: Carácter y código ASCII

Escribir un programa que lea un carácter y muestre por pantalla el carácter ingresado y su código ASCII en formato decimal.

Ejercicio 2: Palabra al revés

Escribir un programa que lea una palabra y la muestre por pantalla en orden inverso. Considere que la palabra ingresada no contiene espacios.

Ejercicio 3: Conversión de texto a mayúsculas

Escribir un programa que lea una cadena de caracteres y la muestre por pantalla convirtiendo todas las letras minúsculas a mayúsculas. Para realizar la conversión, tenga en cuenta que la diferencia entre el código ASCII de una letra minúscula y el de su correspondiente letra mayúscula es un valor constante.

Ejercicio 4: Operaciones con cadenas

Escribir un programa que lea dos cadenas de caracteres y realice las siguientes operaciones:

a. muestre la longitud de cada cadena;

b. compare ambas cadenas e indique si son iguales;

c. concatene la segunda cadena al final de la primera;

d. copie el contenido resultante de la primera cadena en la segunda.

Bloque B: Vectores y matrices

Los siguientes ejercicios utilizan vectores y matrices. Presta especial atención a los índices y a las dimensiones necesarias para cada operación.

Ejercicio 5: Matriz y transpuesta

Escribir un programa que genere una matriz cuadrada M de 3×3 elementos, cuyos valores se calculen mediante la expresión

$$M[i, j] = (i + 2j)^2,$$

considerando que los indices i y j toman valores desde 0 hasta 2. Luego, mostrar por pantalla la matriz M y su transpuesta.

Ejercicio 6: Producto de una matriz por un vector

Escribir un programa que lea una matriz M de 3×3 elementos y un vector v de 3 elementos. Calcular el producto de la matriz por el vector y mostrar por pantalla el vector resultante.

Para cada posición del vector resultado r, considerar:

$$r[i] = \sum_{j=0}^{2} M[i][j] \cdot v[j]$$

Ejercicio 7: Suma de vectores utilizando punteros

Escribir un programa que contenga dos vectores de 5 números enteros y, utilizando punteros, genere un tercer vector cuyos elementos sean la suma de los elementos correspondientes de los dos vectores originales. Luego, mostrar por pantalla el vector resultante.

Bloque C: Punteros y análisis de programas

En este bloque se analizará el comportamiento de programas que utilizan punteros. Antes de ejecutar los fragmentos, intenta seguir los cambios de cada variable y determinar hacia qué objeto apunta cada puntero.

Ejercicio 8: Análisis de programas con punteros y vectores

Antes de ejecutar los programas, analice cada uno de los siguientes códigos e indique qué valores espera que se muestren por pantalla. Luego, compruebe su respuesta compilando y ejecutando cada caso.

a)

```c
int main (void)
{
    int a = 0;
    int *ptr;
    ptr = &a;
    a = a + 3;
    printf ("%d", *ptr);
    return 0;
}
```

b)

```c
int main (void)
{
    int x = 1, y = 2;
    int *ptr;
    ptr = &x;
    y = *ptr;
    *ptr = 0;
    printf ("%d", *ptr);
    printf ("%d", x);
    printf ("%d", y);
    return 0;
}
```

c)

```c
int main (void)
{
    int x = 1, y = 2;
    int *ptr_x, *ptr_y;
    ptr_x = &x;
    ptr_y = &y;
    ++*ptr_x;
    ptr_x = ptr_y;
    printf ("%d", *ptr_x);
    printf ("%d", *ptr_y);
    printf ("%d", x);
    printf ("%d", y);
    return 0;
}
```

d)

```c
int main (void)
{
    int i, j = 0;
    int lista [10];
    for (i = 1; i < 10; i++, j++)
        lista [i] = j++;
    for (i = 5; i < 10; i++)
        printf ("%d\n", lista [i]);
    return 0;
}
```

Ejercicio 9: Operaciones con punteros

Considerando las siguientes declaraciones:

```c
int *pnumero;
int num1, num2;
char *pchar;
char letra1;
num1 = 2;
num2 = 5;
letra1 = 'a';
```

Considere que los incisos se analizan en forma secuencial, manteniendo las asignaciones y modificaciones realizadas en los incisos anteriores.

a. Considerando la instrucción:

```c
pnumero = &num1;
```

Valor de `pnumero`:

I. 2.

II. La dirección de memoria que ocupa `pnumero`.

III. La dirección de memoria que ocupa `num1`.

b. Considerando la instrucción:

```
num2 = num1 + *pnumero;
```

Valor de `num2`:

I. 7.

II. 4.

III. La dirección de `pnumero` más 2.

c. Considerando las instrucciones:

```
pchar = &letra1;
*pchar = 'B';
```

Valor de `letra1`:

I. 'a'.

II. 'B'.

III. La dirección de memoria de `pchar`.

d. ¿Cómo se puede obtener el contenido almacenado en `letra1`?

I. Con `letra1` o `*pchar`.

II. Con `&letra1` o `*pchar`.

III. Con `*letra1` o `&pchar`.

e. Considerando la instrucción:

```
*pnumero = 10;
```

¿Qué efecto produce esta instrucción?

I. Asigna el valor `10` al contenido de `num1`.

II. Asigna la dirección `10` a `pnumero`.

III. Asigna el valor `10` a `num2`.

Bloque D: Ejercicio adicional – Ingeniería en Telecomunicaciones

El último ejercicio aplica el trabajo con cadenas a una situación sencilla de normalización de identificadores en un sistema de monitoreo.

Ejercicio 10: Normalización de nombres de estaciones

Un operador ingresa el nombre de una estación de monitoreo, pero puede escribir algunas o todas sus letras en minúscula. Escriba un programa que:

a. lea una cadena de caracteres con el nombre de la estación;

b. convierta todas las letras minúsculas a mayúsculas (sin utilizar `strupr()`);

c. muestre por pantalla el nombre de la estación normalizado en mayúsculas.

Lista de comprobación final

Antes de finalizar la guía, comprueba que puedes:

- ☐ leer y mostrar un carácter y su código ASCII;
- ☐ recorrer una cadena carácter por carácter;
- ☐ transformar letras minúsculas en mayúsculas;
- ☐ utilizar operaciones básicas de `string.h`;
- ☐ recorrer vectores y matrices mediante índices;
- ☐ obtener y mostrar la transpuesta de una matriz;
- ☐ realizar una operación entre una matriz y un vector;
- ☐ utilizar un puntero para acceder a los elementos de un array;
- ☐ diferenciar dirección de memoria y contenido almacenado;
- ☐ interpretar los operadores de dirección `&` y contenido `*`;
- ☐ analizar un programa con punteros antes de ejecutarlo.